PS 02 IVDC

Vardaan Saxena

October 2026

Gate 1

Path Planning Fundamentals

Question 1: Difference between a *path* and a *trajectory*?
A **path** describes where to go: a sequence of points or grid cells from the start to the goal. It doesn't specify how long the trip takes or how fast to move. A **trajectory** describes where to be and when. It adds timing and motion details, such as speed and sometimes acceleration.
Example: A path might be "go through cells A, B, and C." A trajectory might say "reach B in 2 seconds and C in 4 seconds, moving at these speeds." So, a path planner can find a route; the vehicle then needs a trajectory to follow that route over time.

Question 2: Define "waypoint" , "obstacle" , "free space" , "start state" and "goal" .

- Waypoint - A waypoint can be understood as an intermediate goal or small checkpoints the machine has to reach. For example, if it starts at A and has to reach Z. We can specify that the points B, C, D and so on, which lie between these will be the waypoints.
- Obstacle - A region or a point of the map which is unavailable or unsafe to go through, on the grid we make sure the machine moves around it.
- Free Space - The area the robot is allowed to travel through. On the grid, these are the open cells. It means available for the robot to move through.
- Start State - The start state tells the planner where the robot begins. In our simple grid, that's just its starting cell. "State" is a broader word because a more detailed planner might also track which way the robot is facing or how fast it's moving.

Gate 2

Environment Representation

Question 1: What is a 2D grid map?
A 2D grid map can be thought of as a graph paper put over our image. It can help us identify the roads or lanes as small squares which we can then classify as obstacles etc., we can think of it as a way to help us navigate around the route.

Question 2: What is an occupancy grid?
An occupancy grid can be understood as a grid where each box has info about the surroundings. Whether it may be a wall, a road or an obstacle. Sometimes the information of a grid can also be probabilistic, it might be unknown what's in that specific cell.

Question 3: Explain free, occupied and unknown cells.
As mentioned earlier, the machine can create an occupancy grid where each cell is mentioned as above. A *free* cell will be one where nothing is present right now, it however does not guarantee nothing can move here later. An *occupied* cell means that it is currently occupied by an obstacle. The sensor treats it as blocked and moves around it. An *unknown* cell is one where, as the name suggests, the data is unknown. It might be because of less information right now. A cautious planner may avoid it until it has more information.

Gate 3

Search algorithms

BFS (Breadth-first Search)

Breadth-First Search (BFS) is an algorithm used to find a path from a starting point to a destination. It works by exploring the nearby positions first and then gradually moving toward positions farther away. For example, in a 2D grid, BFS first checks the cells next to the starting cell, then the cells two steps away, then three steps away, and so on until it reaches the destination. It uses a *queue*, which follows the ***First-In, First-Out (FIFO)*** rule, meaning the cell added first is explored first. It also keeps track of the cells it has already visited so that it doesn't check them again. In an occupancy grid, BFS moves through free cells while avoiding cells containing obstacles. Since it explores the closest cells first, it can find the shortest path in terms of the number of steps, as long as every step has the same cost. However, BFS may take more time in large grids because it explores many cells that might not lead toward the destination.

Dijkstra's Algorithm

Dijkstra's Algorithm is a pathfinding algorithm used to find the path with the lowest total cost between a starting point and a destination. Unlike BFS, which

finds the path with the fewest steps, Dijkstra considers the cost (or weight) of each movement. It starts by assigning a distance of 0 to the starting point and infinity to all other points. It then checks the neighboring points and calculates the cost of reaching each one. The algorithm always explores the unvisited point with the lowest known total cost. From that point, it checks the neighboring points again and updates their costs if it finds a cheaper way to reach them. This process continues until the destination is reached. It is important to understand that exploring a point does not mean the algorithm has chosen that route to the destination; it is simply checking possible paths and comparing their costs. In a 2D occupancy grid, Dijkstra can find a path through free cells while avoiding obstacles and considering different movement costs. It guarantees the lowest-cost path as long as all movement costs are non-negative, although it may explore many unnecessary cells before reaching the goal.

A^* Algorithm

(A-Star) Algorithm is a path-finding algorithm used to find the path with the lowest total cost between a starting point and a destination. It is similar to Dijkstra's algorithm, but instead of only considering the cost of reaching a point, it also estimates how much it would cost to reach the destination from that point. It uses the formula f(n) = g(n) + h(n), where g(n) is the cost of reaching the current point from the start, h(n) is the estimated remaining cost to the goal, and f(n) is the estimated total cost of the path. This estimate is calculated using a heuristic, such as Manhattan distance or straight-line distance. The algorithm explores the point with the lowest f(n) value first and updates the costs whenever it discovers a cheaper route. Even if the estimate is not completely accurate, A* can still find the shortest path as long as the heuristic does not overestimate the actual remaining cost and the algorithm is implemented correctly. In a 2D occupancy grid, A* can find a path through free cells while avoiding obstacles. Since it uses an estimate of the distance to the goal, it can often find the shortest path while exploring fewer cells than Dijkstra's algorithm.

Gate 4

Global and Local Planning

Question 1: Difference between a **global path** and a **local path**?
An autonomous vehicle needs to decide two things in order to reach from the start to the goal. The first one is the *global planning* which determines the complete map of the route, sets the waypoints and uses a search algorithm mentioned previously. Secondly, we have the *local planning* which can determine changes in the path which may not be planned ahead of the time. For example, if a pedestrian or an unexpected obstacle appears on the planned route, the local planner can decide whether the vehicle should slow down, stop, or move around it safely. While global planning focuses on finding a suitable route to the destination, local planning focuses on following that route safely while responding

to changes in the environment. Both work together to ensure that the vehicle reaches its destination efficiently and safely.

Question 2: Why an autonomous vehicle may need to re-plan when a new obstacle appears.

An autonomous vehicle may need to re-plan its path when a new obstacle appears because its previously calculated route may no longer be safe or accessible. For example, imagine a robot travelling along a path calculated using A*. Suddenly, a person or another vehicle blocks its route. The robot cannot simply continue following the original path, as doing so could cause a collision. Instead, it uses its sensors to detect the obstacle and updates its understanding of the surroundings. Its local planner may decide to slow down, stop, or move around the obstacle if a safe alternative is available. However, if the obstacle completely blocks the planned route, the vehicle may need to use a pathfinding algorithm again to calculate a new global path. This process is called re-planning, and it allows the vehicle to adapt to changes in its environment while continuing toward its destination.

Thank you